



EVIDENCIA GA7-220501096-AA5-EV02

API – TESTING CON POSTMAN

PRESENTADO POR:

ALISON JULIETH GALVIS DURÁN

INSTRUCTOR:

EDDUAR YAKSEIR PEREZ PEREZ

SERVICIO NACIONAL DE APRENDIZAJE – SENA

TECNOLOGÍA EN ANÁLISIS Y DESARROLLO DE SOFTWARE (ADSO)

FICHA: 3235878

SEPTIEMBRE DE 2026

Introducción

En la evidencia GA7-220501096-AA5-EV01 se desarrolló AYB Auth API, un servicio web REST en Java con Spring Boot que permite registrar usuarios e iniciar sesión. El servicio responde "Autenticación satisfactoria" cuando el usuario y la contraseña son correctos y "Error en la autenticación" cuando no lo son.

En esta evidencia se realiza el testing de ese servicio con Postman. Se diseñaron 14 casos de prueba, que cubren tanto el funcionamiento esperado como el manejo de errores, y se automatizaron 54 validaciones con scripts de Postman. El documento presenta el entorno, la instalación de la herramienta, los endpoints, el plan de pruebas, la evidencia de cada ejecución y las conclusiones.

Objetivo general

Verificar mediante pruebas funcionales con Postman el correcto funcionamiento del servicio web de registro e inicio de sesión desarrollado en la AA5-EV01.

Objetivos específicos

- Instalar Postman y configurar una colección y un entorno para la API.
- Diseñar casos de prueba positivos y negativos para los endpoints de registro e inicio de sesión.
- Automatizar la verificación de códigos de estado, mensajes y tiempos de respuesta con scripts de prueba.
- Documentar con pantallazos los resultados y entregar los endpoints de la API.

Herramientas y entorno de pruebas

Herramienta	Versión	Uso en la evidencia
Java (JDK)	17	Lenguaje de programación del servicio
Spring Boot	3.4.5	Framework del servicio web REST (Spring Web)
Spring Data JPA	3.4.x	Acceso a datos mediante UsuarioRepository
H2 Database	2.3.232	Base de datos en archivo (./data/aybdb)
Bean Validation	3.x	Validación de campos (@NotBlank, @Size)
BCrypt (Spring Security Crypto)	6.x	Cifrado de contraseñas
Maven	3.9	Construcción y ejecución del proyecto
Postman	Última versión (escritorio)	Herramienta de testing de la API
Git / GitHub	GitHub Desktop 64 bits.	Control de versiones del proyecto

Instalación de Postman

- Ingresar a <https://www.postman.com/downloads/> y descargar el instalador para Windows.
- Ejecutar el instalador y esperar a que finalice la instalación.
- Abrir Postman e iniciar sesión con una cuenta gratuita.
- Crear un espacio de trabajo (Workspace) para la evidencia.

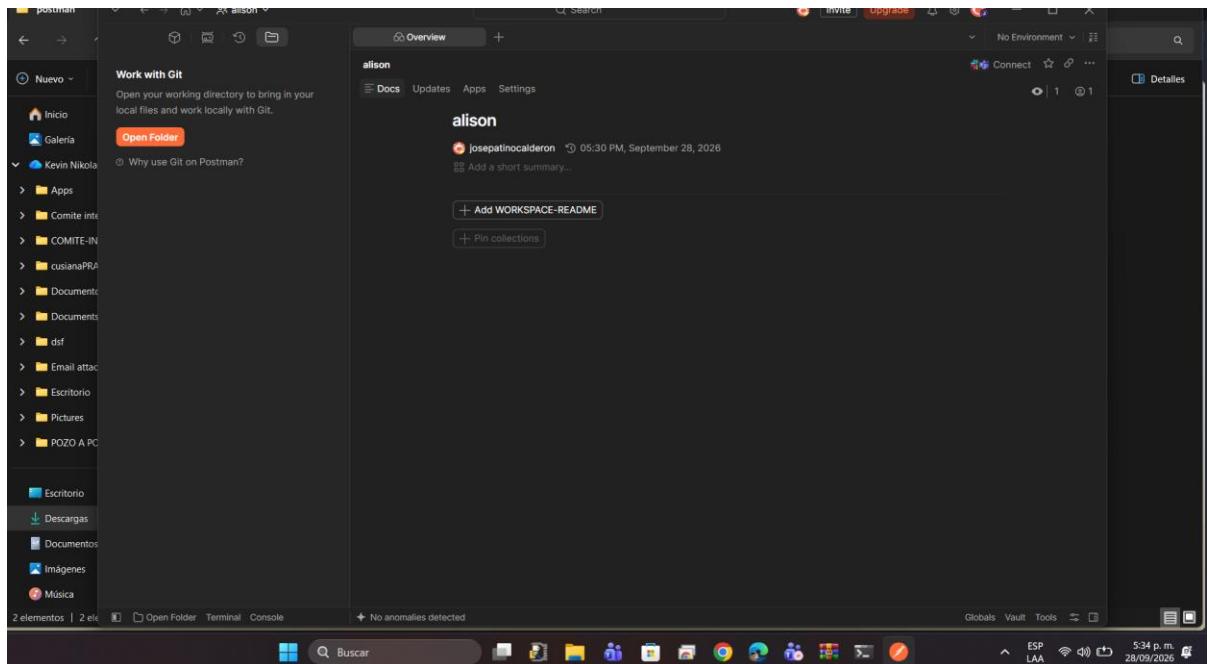


Figura 1. Postman de escritorio instalado y abierto, con el workspace creado para la evidencia.

Descripción de la API

El proyecto sigue una arquitectura por capas: controller (AuthController, recibe las peticiones HTTP), service (AuthService, lógica de registro y autenticación), repository (UsuarioRepository, acceso a la base de datos), model (entidad Usuario), dto (AuthRequest y ApiResponse) y config (bean de BCrypt). La URL base es <http://localhost:8080>.

Método	Endpoint	Descripción
POST	/api/auth/registro	Registra un usuario nuevo. Responde 201, 409 si el usuario ya existe o 400 si los datos no son válidos.
POST	/api/auth/login	Valida usuario y contraseña. Responde 200 "Autenticación satisfactoria", 401 "Error en la autenticación" o 400 si faltan datos.

Ambos endpoints reciben un JSON con los campos usuario y password, y responden con un JSON con los campos success y message. El detalle completo está en el archivo ENDPOINTS_API incluido en la entrega.

Configuración de Postman

Ejecutar la API

En la carpeta del proyecto se ejecuta mvn spring-boot:run (o el botón Run del IDE) y se verifica en consola el mensaje "Tomcat started on port 8080".

```

: No LoadTimeWeaver setup: ignoring JPA class transformer
2026-09-28T13:49:06.412-05:00 INFO 24792 --- [ayb-auth-api] [
: HikariPool-1 - Starting...
2026-09-28T13:49:06.613-05:00 INFO 24792 --- [ayb-auth-api] [
: HikariPool-1 - Added connection conn0: url=jdbc:h2:file:./data/aybdb user=SA
2026-09-28T13:49:06.616-05:00 INFO 24792 --- [ayb-auth-api] [
: HikariPool-1 - Start completed.
2026-09-28T13:49:06.676-05:00 INFO 24792 --- [ayb-auth-api] [
: HHH10001005: Database info:
Database JDBC URL [Connecting through datasource 'HikariDataSource (HikariPool-1)']
Database driver: undefined/unknown
Database version: 2.3.232
Autocommit mode: undefined/unknown
Isolation level: undefined/unknown
Minimum pool size: undefined/unknown
Maximum pool size: undefined/unknown
2026-09-28T13:49:07.422-05:00 INFO 24792 --- [ayb-auth-api] [
: HHH000489: No JTA platform available (set 'hibernate.transaction.jta.platform' to enable JTA platform integration)
2026-09-28T13:49:07.552-05:00 INFO 24792 --- [ayb-auth-api] [
: Initialized JPA EntityManagerFactory for persistence unit 'default'
2026-09-28T13:49:07.950-05:00 WARN 24792 --- [ayb-auth-api] [
: spring.jpa.open-in-view is enabled by default. Therefore, database queries may be performed during view rendering. Exp
licitly configure spring.jpa.open-in-view to disable this warning
2026-09-28T13:49:08.201-05:00 INFO 24792 --- [ayb-auth-api] [
: H2 console available at '/h2-console'. Database available at 'jdbc:h2:file:./data/aybdb'
2026-09-28T13:49:08.282-05:00 INFO 24792 --- [ayb-auth-api] [
: Tomcat started on port 8080 (http) with context path '/'
2026-09-28T13:49:08.295-05:00 INFO 24792 --- [ayb-auth-api] [
: Started AybAuthApiApplication in 4.579 seconds (process running for 4.947)

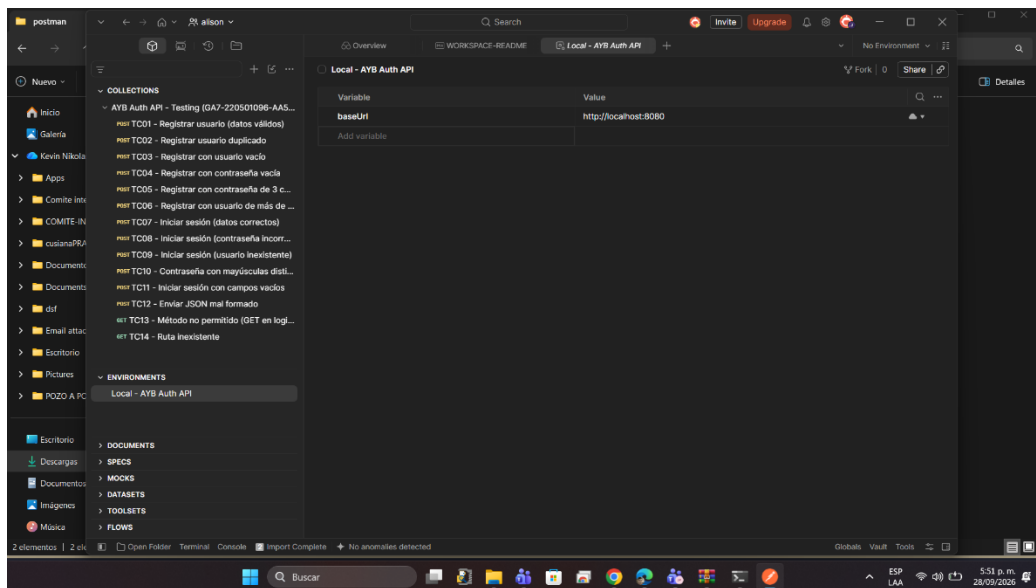
```

Importar la colección y el entorno

En Postman hacer clic en Import y seleccionar los archivos de la carpeta postman/:
 AYB_Auth_API.postman_collection.json y AYB_Auth_API.postman_environment.json.

Seleccionar el entorno Local - AYB Auth API en la esquina superior derecha.

Variable	Ámbito	Descripción
{{baseUrl}}	Entorno	URL base de la API: http://localhost:8080
{{usuarioPrueba}}	Colección	Usuario único generado en TC01 (alison_XXXXXX)
{{passwordPrueba}}	Colección	Contraseña del usuario de prueba (Clave123)



Scripts de prueba automatizados

Cada petición tiene en la pestaña Scripts › Post-response validaciones escritas con la librería pm de Postman. Por ejemplo, el caso de login correcto (TC07):

Todas las peticiones verifican además que la respuesta sea JSON y que el tiempo de respuesta sea menor a 2000 ms.

Plan de pruebas

Se definieron 14 casos de prueba: 2 positivos y 12 negativos. Se ejecutan en orden, porque TC02, TC07, TC08 y TC10 usan el usuario creado en TC01.

ID	Caso de prueba	Método	Endpoint	Tipo	Código esperado
TC01	Registrar usuario (datos válidos)	POST	/api/auth/registro	Positivo	201 Created
TC02	Registrar usuario duplicado	POST	/api/auth/registro	Negativo	409 Conflict
TC03	Registrar con usuario vacío	POST	/api/auth/registro	Negativo	400 Bad Request
TC04	Registrar con contraseña vacía	POST	/api/auth/registro	Negativo	400 Bad Request

TC05	Registrar con contraseña de 3 caracteres	POST	/api/auth/registro	Negativo	400 Bad Request
TC06	Registrar con usuario de más de 80 caracteres	POST	/api/auth/registro	Negativo	400 Bad Request
TC07	Iniciar sesión (datos correctos)	POST	/api/auth/login	Positivo	200 OK
TC08	Iniciar sesión (contraseña incorrecta)	POST	/api/auth/login	Negativo	401 Unauthorized
TC09	Iniciar sesión (usuario inexistente)	POST	/api/auth/login	Negativo	401 Unauthorized
TC10	Contraseña con mayúsculas distintas	POST	/api/auth/login	Negativo	401 Unauthorized
TC11	Iniciar sesión con campos vacíos	POST	/api/auth/login	Negativo	400 Bad Request

TC12	Enviar JSON mal formado	POST	/api/auth/login	Negativo	400 Bad Request
TC13	Método no permitido (GET en login)	GET	/api/auth/login	Negativo	405 Method Not Allowed
TC14	Ruta inexistente	GET	/api/auth/perfil	Negativo	404 Not Found

Ejecución de las pruebas y evidencias

Para cada caso se presenta el objetivo, la petición enviada, el resultado esperado, las validaciones automatizadas, el resultado obtenido y el pantallazo de Postman con la respuesta.

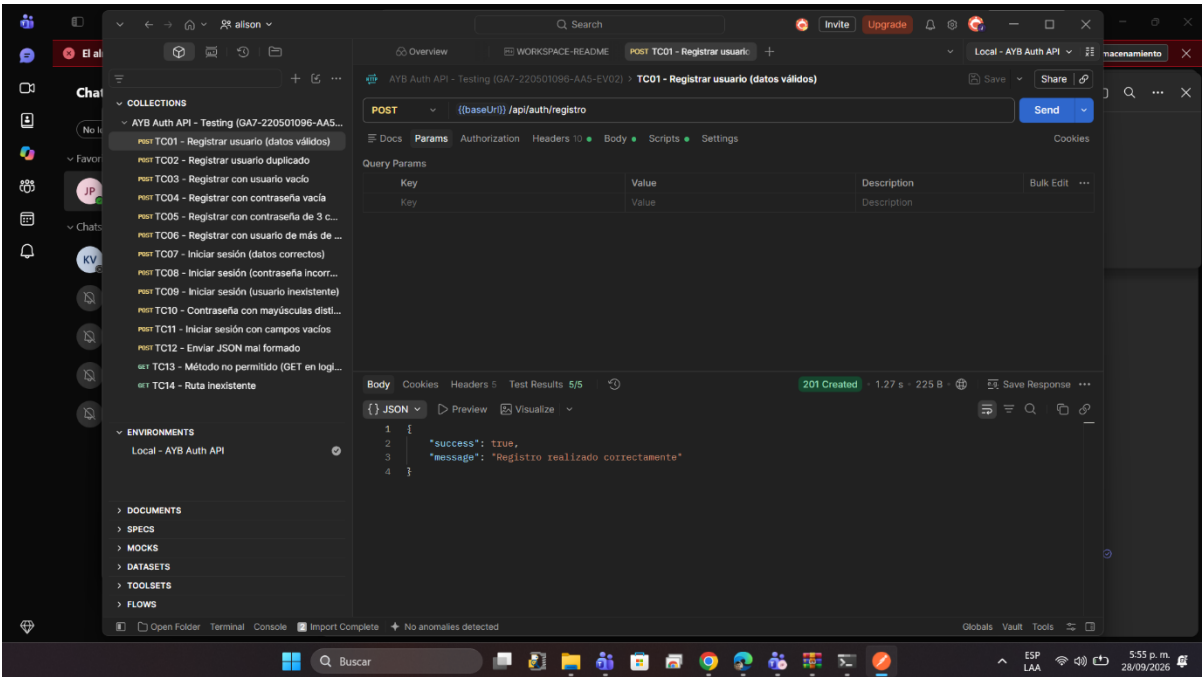
TC01 – Registrar usuario (datos válidos)

Objetivo: Comprobar que se puede registrar un usuario nuevo y que la contraseña se guarda cifrada con BCrypt. Un pre-request script genera un nombre de usuario único (alison_XXXXXX) para que la colección pueda repetirse, ya que la base H2 guarda los datos en disco.

Método y URL	POST http://localhost:8080/api/auth/registro
Cuerpo (JSON)	<pre>{ "usuario": "{{usuarioPrueba}}", "password": "{{passwordPrueba}}" }</pre>
Resultado esperado	201 Created – success = true, "Registro realizado correctamente"



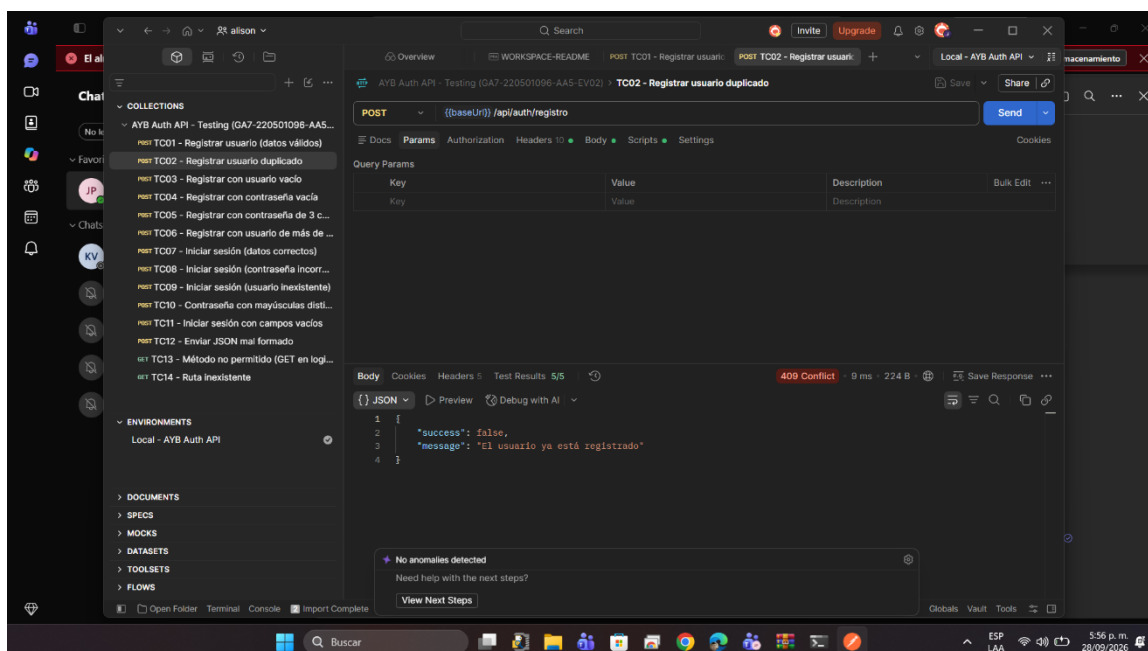
Validaciones automáticas	✓ Código de estado 201 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms ✓ success = true ✓ Mensaje: Registro realizado correctamente
Resultado obtenido	Código: 201 Created Tiempo: 1270 ms Test Results: 5/5 Estado: ☒ Aprobado ☐ Fallido



TC02 – Registrar usuario duplicado

Objetivo: Comprobar que la API no permite registrar dos veces el mismo usuario (método existsByUsuario y restricción unique de la entidad).

Método y URL	POST http://localhost:8080/api/auth/registro
Cuerpo (JSON)	<pre>{ "usuario": "{{usuarioPrueba}}", "password": "{{passwordPrueba}}" }</pre>
Resultado esperado	409 Conflict – success = false, "El usuario ya está registrado"
Validaciones automáticas	✓ Código de estado 409 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms ✓ success = false ✓ Mensaje: El usuario ya está registrado
Resultado obtenido	Código: 409 Conflict Tiempo: 9 ms Test Results: 5/5 Estado: ☒ Aprobado ☐ Fallido

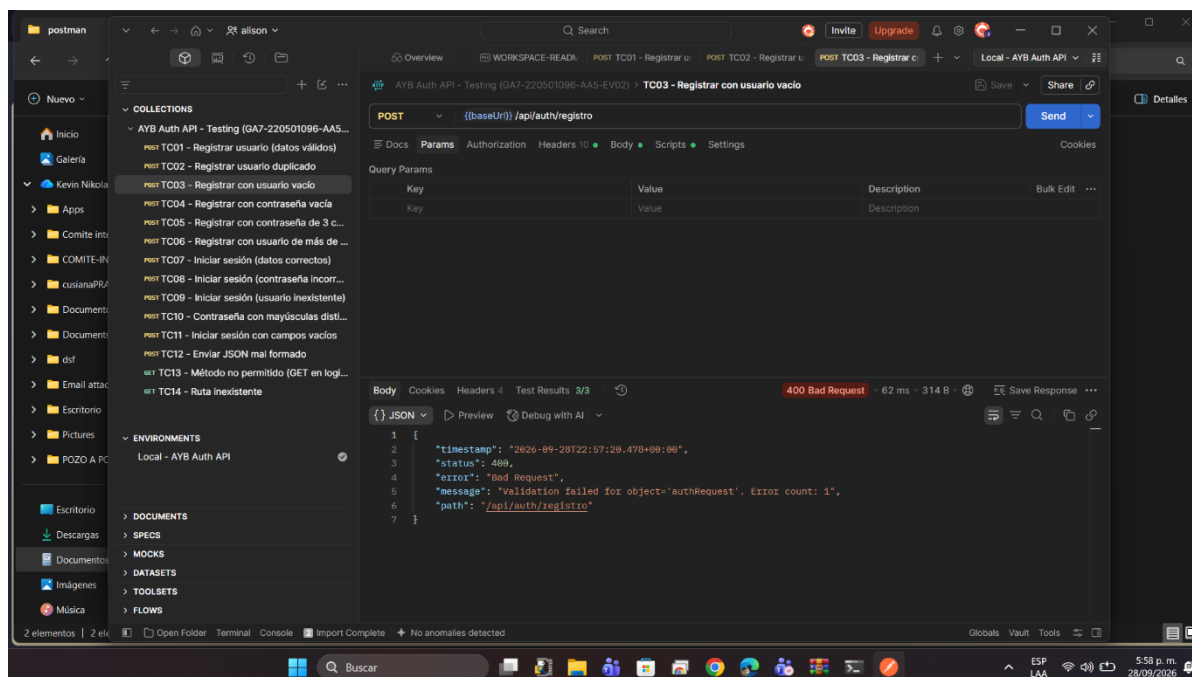


TC03 – Registrar con usuario vacío

Objetivo: Comprobar la validación @NotBlank del campo usuario.

Método y URL	POST http://localhost:8080/api/auth/registro
Cuerpo (JSON)	<pre>{ "usuario": "", "password": "123456" }</pre>
Resultado esperado	400 Bad Request – Error de validación: usuario obligatorio
Validaciones automáticas	✓ Código de estado 400 ✓ La respuesta es JSON

	✓ Tiempo de respuesta menor a 2000 ms
Resultado obtenido	Código: 400 Bad Request Tiempo: 62 ms Test Results: 3/3 Estado: ☒ Aprobado ☐ Fallido

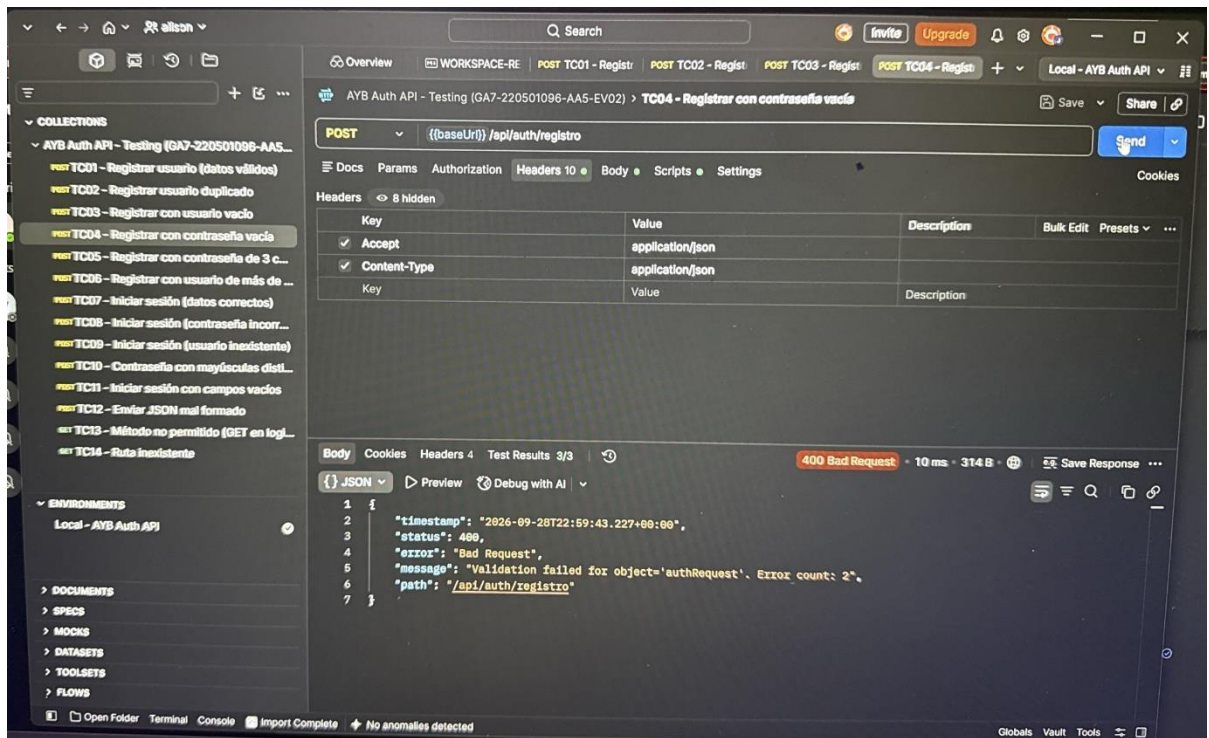


TC04 – Registrar con contraseña vacía

Objetivo: Comprobar la validación @NotBlank del campo password.

Método y URL	POST http://localhost:8080/api/auth/registro
Cuerpo (JSON)	<pre>{ "usuario": "prueba_vacia", "password": "" }</pre>

Resultado esperado	400 Bad Request – Error de validación: contraseña obligatoria
Validaciones automáticas	✓ Código de estado 400 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms
Resultado obtenido	Código: 400 Bad Request Tiempo: 10 ms Test Results: 3/3 Estado: ☒ Aprobado ☐ Fallido



The screenshot shows the Postman interface for a test titled "TC04 - Registrar con contraseña vacía". The test is a POST request to the endpoint `/(baseUri)/api/auth/registro` with headers `Accept: application/json` and `Content-Type: application/json`. The test results show a "400 Bad Request" status with a response time of 10 ms and a body containing the following JSON:

```

{
  "timestamp": "2026-09-26T22:59:43.227+00:00",
  "status": 400,
  "error": "Bad Request",
  "message": "Validation failed for object='authRequest'. Error count: 2.",
  "path": "/api/auth/registro"
}

```

The left sidebar shows a collection of tests for the "AYB Auth API - Testing" workspace, including TC01 through TC14. The bottom status bar indicates "No anomalies detected".

TC05 – Registrar con contraseña de 3 caracteres

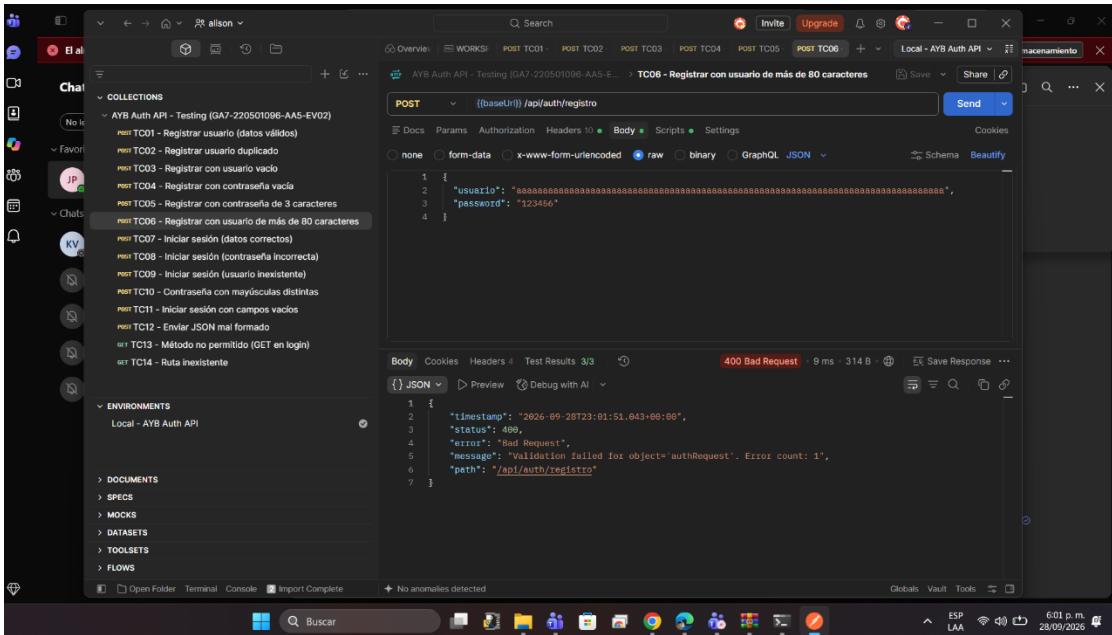
Objetivo: Comprobar la validación @Size(min = 4) de la contraseña.

Método y URL	POST http://localhost:8080/api/auth/registro
Cuerpo (JSON)	<pre>{ "usuario": "prueba_corta", "password": "123" }</pre>
Resultado esperado	400 Bad Request – Error de validación: contraseña entre 4 y 100 caracteres
Validaciones automáticas	✓ Código de estado 400 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms
Resultado obtenido	Código: 400 Bad Request Tiempo: 12 ms Test Results: 3/3 Estado: ☒ Aprobado ☐ Fallido

Objetivo: Comprobar la validación @Size(max = 80) del usuario.

Método y URL	POST http://localhost:8080/api/auth/registro
Cuerpo (JSON)	<pre>{ "usuario": "aa...", "password": "123456" }</pre>
Resultado esperado	400 Bad Request – Error de validación: usuario máximo 80 caracteres
Validaciones automáticas	✓ Código de estado 400 ✓ La respuesta es JSON

	✓ Tiempo de respuesta menor a 2000 ms
Resultado obtenido	Código: 400 Bad Request Tiempo: 9 ms Test Results: 3/3 Estado: ☒ Aprobado ☐ Fallido

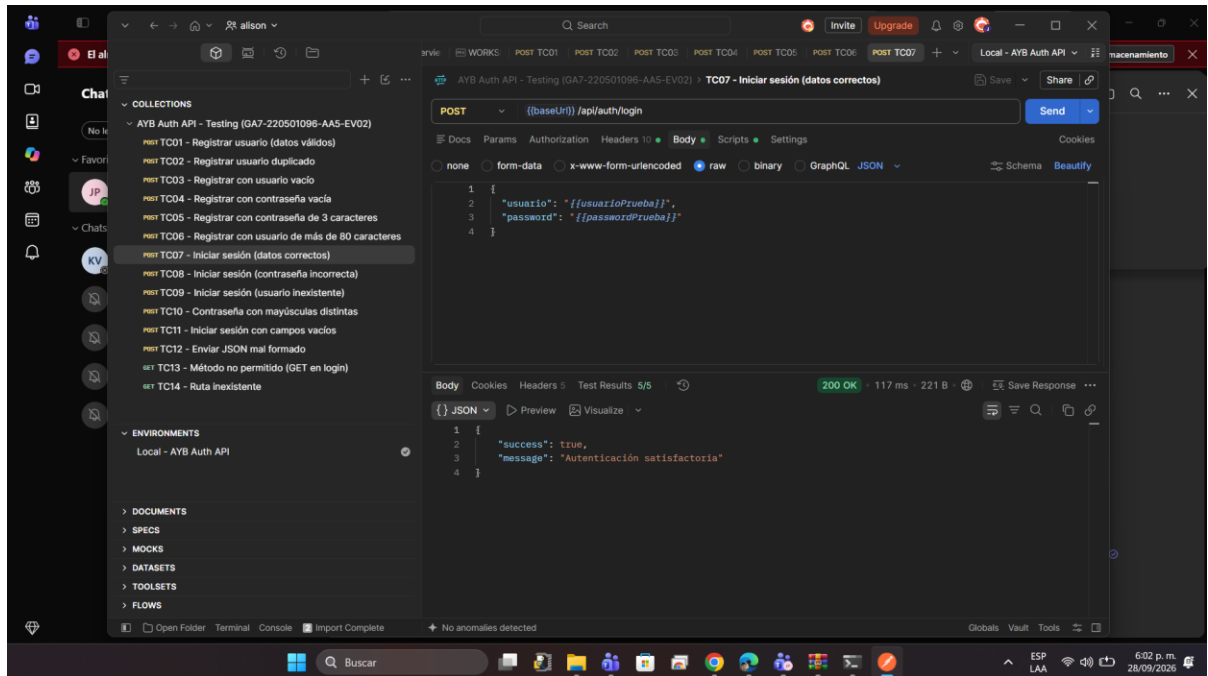


TC07 – Iniciar sesión (datos correctos)

Objetivo: Comprobar que con usuario y contraseña correctos la API responde "Autenticación satisfactoria".

Método y URL	POST http://localhost:8080/api/auth/login
---------------------	--

Cuerpo (JSON)	<pre>{ "usuario": "{{usuarioPrueba}}", "password": "{{passwordPrueba}}" }</pre>
Resultado esperado	200 OK – success = true, "Autenticación satisfactoria"
Validaciones automáticas	✓ Código de estado 200 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms ✓ success = true ✓ Mensaje: Autenticación satisfactoria
Resultado obtenido	Código: 200 OK Tiempo: 117 ms Test Results: 5/5 Estado: ☒ Aprobado ☐ Fallido

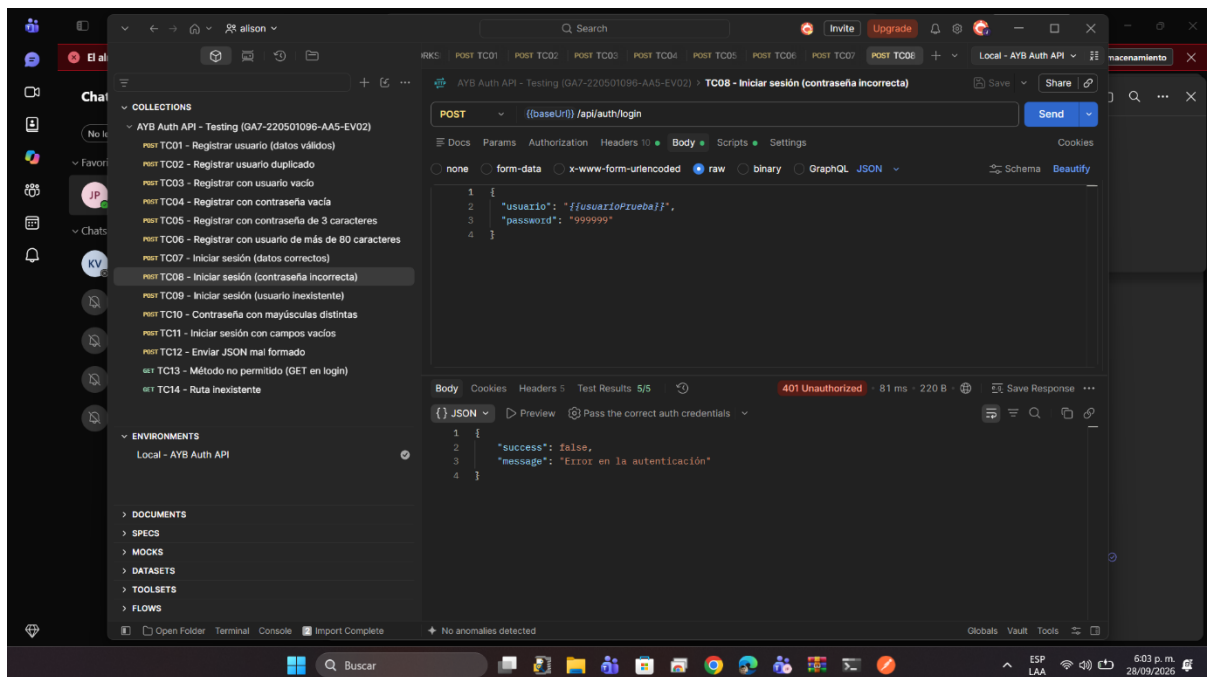


TC08 – Iniciar sesión (contraseña incorrecta)

Objetivo: Comprobar que con contraseña incorrecta la API responde "Error en la autenticación" con código 401.

Método y URL	POST http://localhost:8080/api/auth/login
Cuerpo (JSON)	<pre>{ "usuario": "{{usuarioPrueba}}", "password": "999999" }</pre>
Resultado esperado	401 Unauthorized – success = false, "Error en la autenticación"

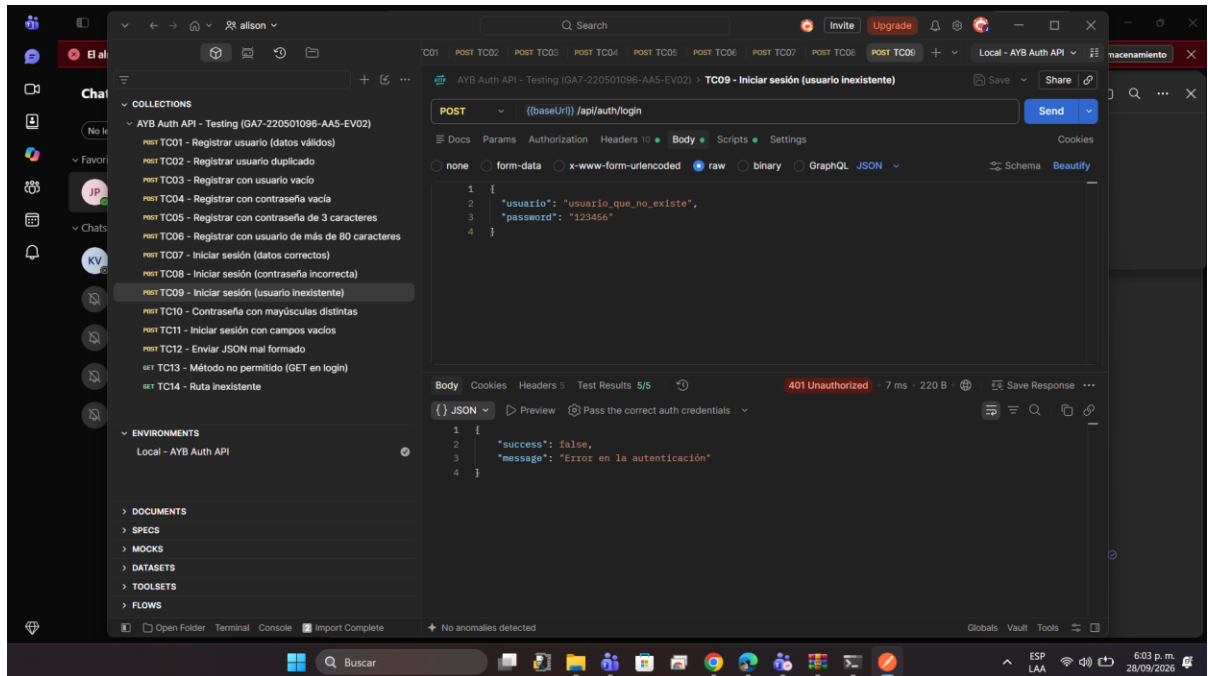
Validaciones automáticas	✓ Código de estado 401 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms ✓ success = false ✓ Mensaje: Error en la autenticación
Resultado obtenido	Código: 401 Unauthorized Tiempo: 81 ms Test Results: 5/5 Estado: ☒ Aprobado ☐ Fallido



TC09 – Iniciar sesión (usuario inexistente)

Objetivo: Comprobar que un usuario no registrado recibe el mismo mensaje de error, sin revelar si el usuario existe.

Método y URL	POST http://localhost:8080/api/auth/login
Cuerpo (JSON)	<pre>{ "usuario": "usuario_que_no_existe", "password": "123456" }</pre>
Resultado esperado	401 Unauthorized – success = false, "Error en la autenticación"
Validaciones automáticas	✓ Código de estado 401 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms ✓ success = false ✓ Mensaje: Error en la autenticación
Resultado obtenido	Código: 401 Unauthorized Tiempo: 7 ms Test Results: 5/5 Estado: ☒ Aprobado ☐ Fallido

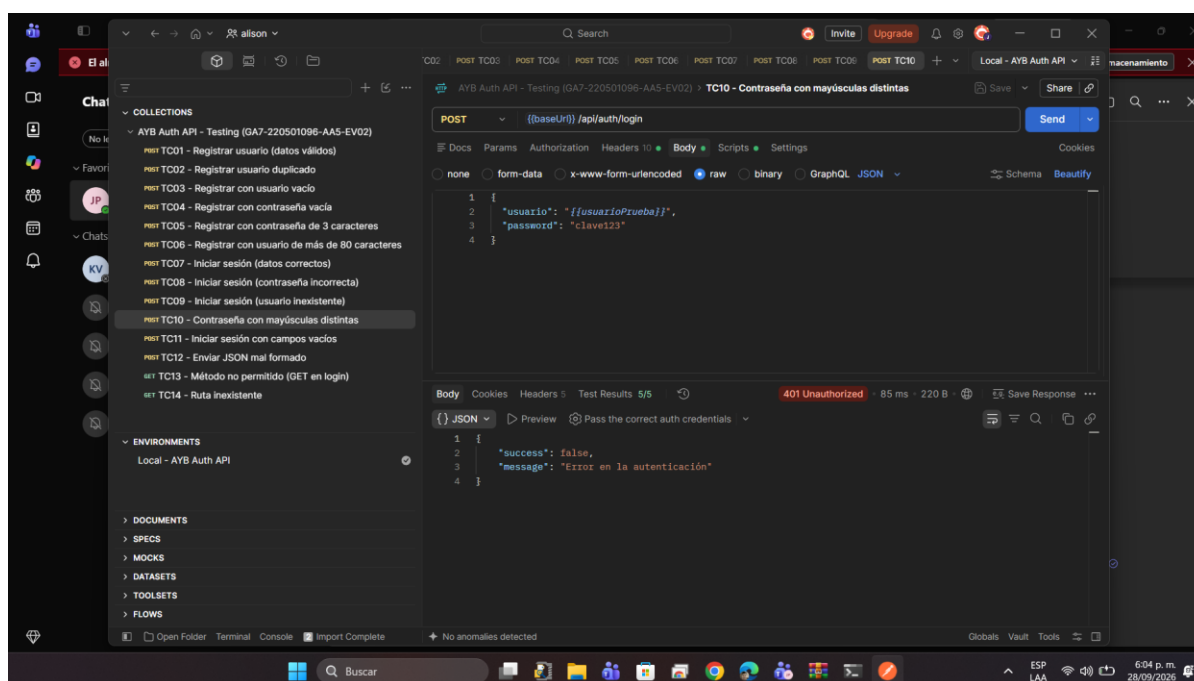


TC10 – Contraseña con mayúsculas distintas

Objetivo: Comprobar que la contraseña distingue mayúsculas y minúsculas (BCrypt compara el valor exacto).

Método y URL	POST http://localhost:8080/api/auth/login
Cuerpo (JSON)	<pre>{ "usuario": "{{usuarioPrueba}}", "password": "clave123" }</pre>
Resultado esperado	401 Unauthorized – success = false, "Error en la autenticación"
Validaciones automáticas	✓ Código de estado 401 ✓ La respuesta es JSON

	✓ Tiempo de respuesta menor a 2000 ms ✓ success = false ✓ Mensaje: Error en la autenticación
Resultado obtenido	Código: 401 Unauthorized Tiempo: 85 ms Test Results: 5/5 Estado: ☒ Aprobado ☐ Fallido

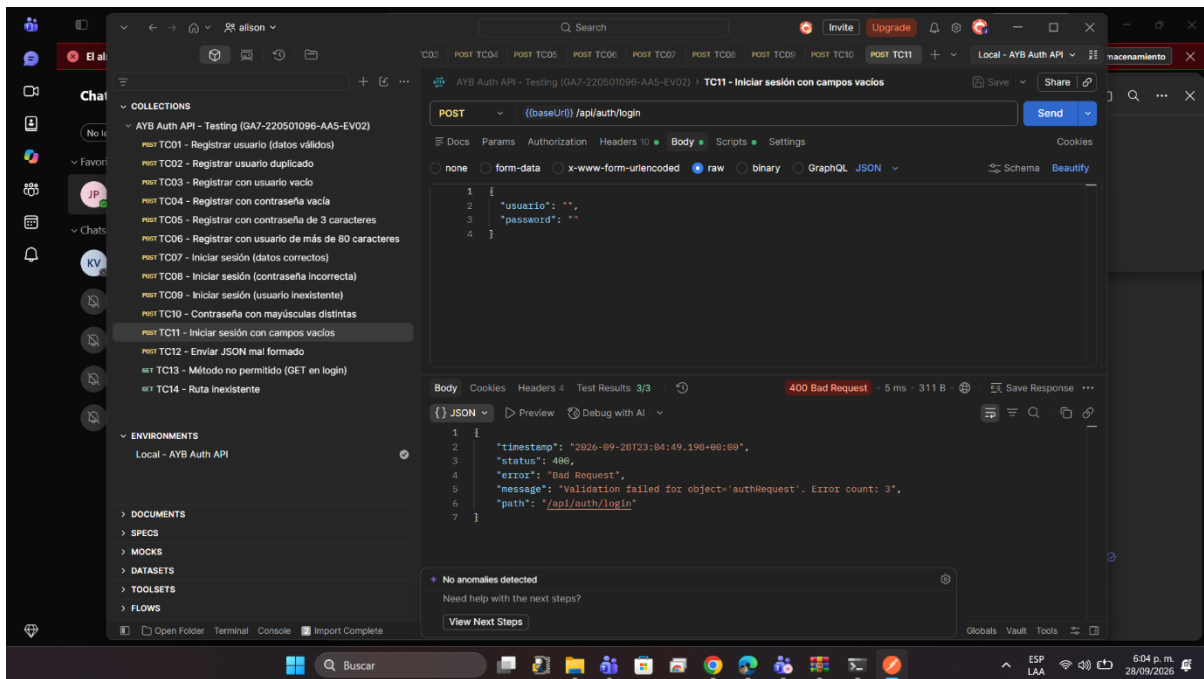


TC11 – Iniciar sesión con campos vacíos

Objetivo: Comprobar que el login también valida los campos obligatorios antes de consultar la base de datos.

Método y URL	POST http://localhost:8080/api/auth/login
Cuerpo (JSON)	<pre>{ "usuario": "", </pre>

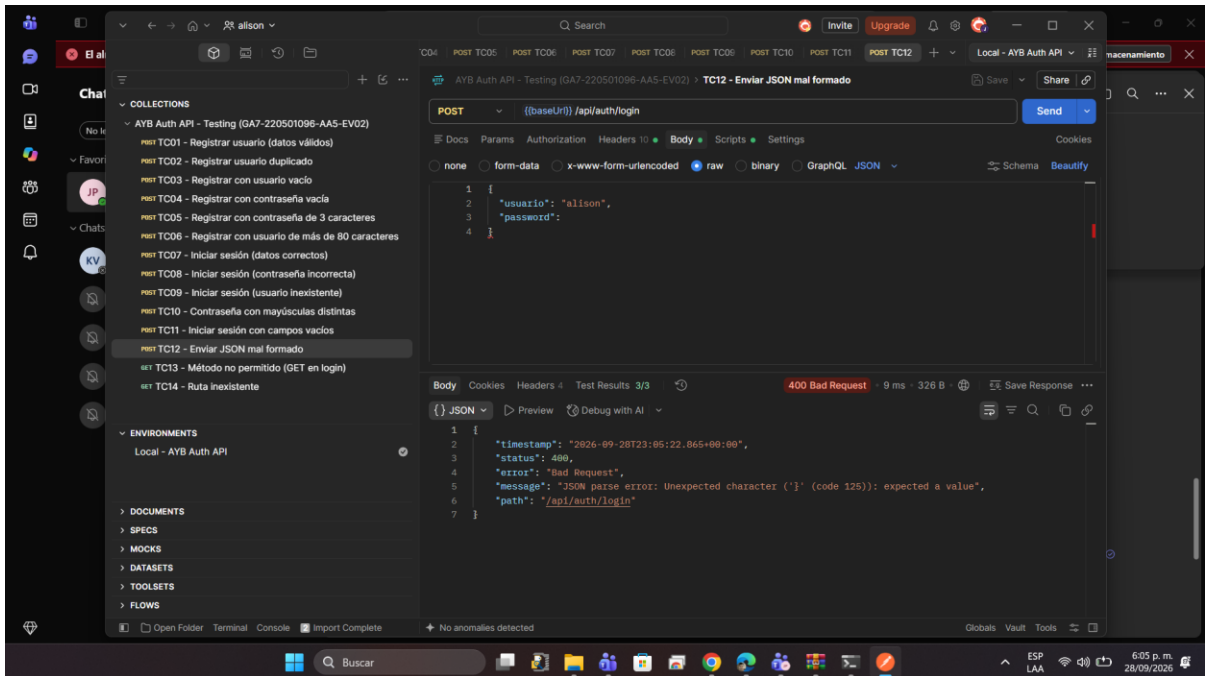
	<pre>"password": "" }</pre>
Resultado esperado	400 Bad Request – Error de validación de campos obligatorios
Validaciones automáticas	✓ Código de estado 400 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms
Resultado obtenido	Código: 400 Bad Request Tiempo: 5 ms Test Results: 3/3 Estado: ☒ Aprobado ☐ Fallido



TC12 – Enviar JSON mal formado

Objetivo: Comprobar que la API responde de forma controlada ante un JSON con errores de sintaxis.

Método y URL	POST http://localhost:8080/api/auth/login
Cuerpo (JSON)	<pre>{ "usuario": "alison", "password": }</pre>
Resultado esperado	400 Bad Request – La API rechaza el cuerpo mal formado
Validaciones automáticas	✓ Código de estado 400 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms
Resultado obtenido	Código: 400 Bad Request Tiempo: 9 ms Test Results: 3/3 Estado: ☒ Aprobado ☐ Fallido

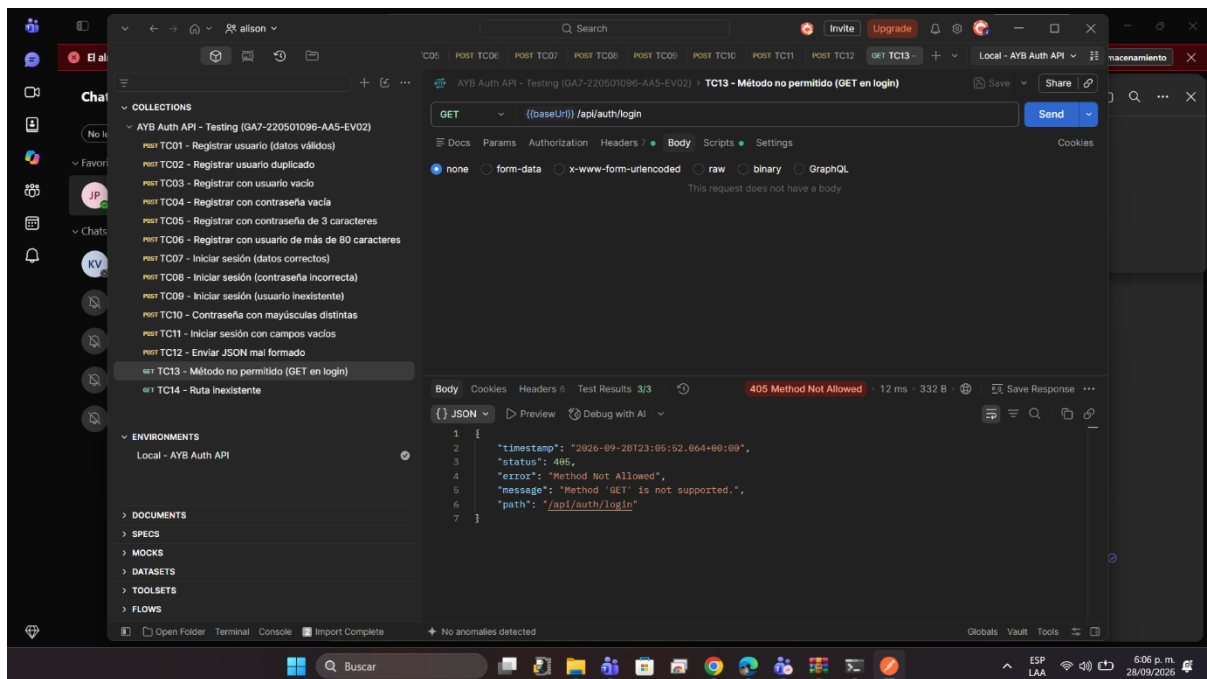


TC13 – Método no permitido (GET en login)

Objetivo: Comprobar que el endpoint de login solo acepta el método POST.

Método y URL	GET http://localhost:8080/api/auth/login
Cuerpo (JSON)	Sin cuerpo
Resultado esperado	405 Method Not Allowed – El endpoint solo acepta POST

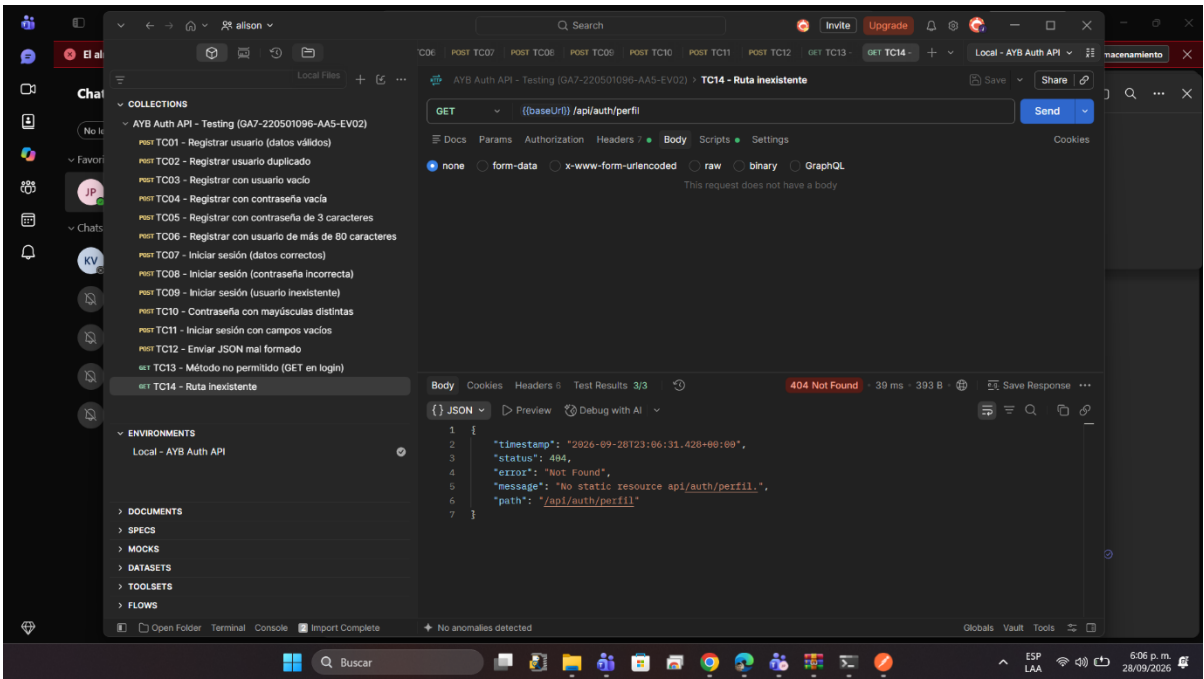
Validaciones automáticas	✓ Código de estado 405 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms
Resultado obtenido	Código: 405 Method Not Allowed Tiempo: 12 ms Test Results: 3/3 Estado: ☒ Aprobado ☐ Fallido



TC14 – Ruta inexistente

Objetivo: Comprobar que una ruta no definida devuelve 404.

Método y URL	GET http://localhost:8080/api/auth/perfil
Cuerpo (JSON)	Sin cuerpo
Resultado esperado	404 Not Found – La ruta no existe
Validaciones automáticas	✓ Código de estado 404 ✓ La respuesta es JSON ✓ Tiempo de respuesta menor a 2000 ms
Resultado obtenido	Código: 404 Not Found Tiempo: 39 ms Test Results: 3/3 Estado: ☒ Aprobado ☐ Fallido

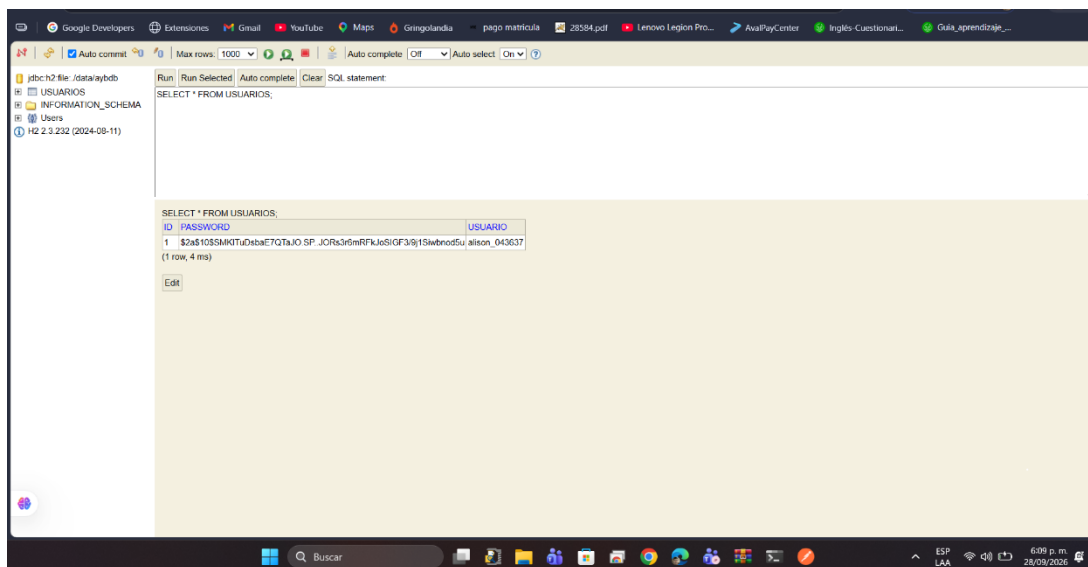


Verificación del cifrado de contraseñas

Para comprobar que la contraseña no se guarda en texto plano se abre la consola de H2 en <http://localhost:8080/h2-console> con la JDBC URL `jdbc:h2:file:./data/aybdb`, usuario `sa` y sin contraseña, y se ejecuta:

```
SELECT * FROM USUARIOS;
```

Resultado obtenido: la tabla `USUARIOS` contiene el usuario de prueba **alison_043637** creado en TC01, y la columna `PASSWORD` muestra un hash BCrypt (texto que empieza por `$2a$10$`) y no la contraseña original.



Resumen de resultados

Se ejecutaron en orden los 14 casos de la colección desde Postman. La siguiente tabla resume los resultados obtenidos en la sección 8.

ID	Código esperado	Código obtenido	Tiempo	Test Results	Estado
TC01	201 Created	201 Created	1270 ms	5/5	Aprobado
TC02	409 Conflict	409 Conflict	9 ms	5/5	Aprobado
TC03	400 Bad Request	400 Bad Request	62 ms	3/3	Aprobado
TC04	400 Bad Request	400 Bad Request	10 ms	3/3	Aprobado
TC05	400 Bad Request	400 Bad Request	12 ms	3/3	Aprobado
TC06	400 Bad Request	400 Bad Request	9 ms	3/3	Aprobado
TC07	200 OK	200 OK	117 ms	5/5	Aprobado
TC08	401 Unauthorized	401 Unauthorized	81 ms	5/5	Aprobado
TC09	401 Unauthorized	401 Unauthorized	7 ms	5/5	Aprobado
TC10	401 Unauthorized	401 Unauthorized	85 ms	5/5	Aprobado

TC11	400 Bad Request	400 Bad Request	5 ms	3/3	Aprobado
TC12	400 Bad Request	400 Bad Request	9 ms	3/3	Aprobado
TC13	405 Method Not Allowed	405 Method Not Allowed	12 ms	3/3	Aprobado
TC14	404 Not Found	404 Not Found	39 ms	3/3	Aprobado

Indicador	Valor
Casos de prueba ejecutados	14
Validaciones automatizadas	54 (54 aprobadas)
Casos aprobados	14 de 14
Casos fallidos	0
Porcentaje de éxito	100 %

Conclusiones

- El servicio cumple con el requerimiento principal: registra usuarios y responde "Autenticación satisfactoria" o "Error en la autenticación" según las credenciales recibidas.
- Las pruebas negativas permiten verificar que la API valida los datos de entrada y responde con los códigos HTTP adecuados (400, 401, 404, 405 y 409) en lugar de fallar.
- Se comprobó la seguridad de la autenticación: las contraseñas se almacenan cifradas con BCrypt, se distinguen mayúsculas y minúsculas, y el mismo mensaje de error se usa para usuario inexistente o contraseña incorrecta.
- Los 14 casos se aprobaron y todos los tiempos de respuesta estuvieron por debajo del límite de 2000 ms (el mayor fue el registro inicial, 1270 ms, que incluye el cifrado BCrypt).
- Los scripts y variables de Postman permitieron automatizar las pruebas, de modo que la colección puede volver a ejecutarse cada vez que se modifique el código (pruebas de regresión).
- Como mejora se recomienda agregar un manejador global de excepciones (`@RestControllerAdvice`) para que los errores de validación (400) respondan con el mismo formato `success/message` que el resto de la API.